오즈코딩스쿨 × 고용노동부 · 1인 창업가 개발부트캠프 7기

JavaScript 기초

DAY 17

비동기 처리 기초

기다리는 일은 맡겨 두고 다음 줄로 넘어가는 비동기 —
setTimeout · Promise · async/await로 실행 순서를 예측합니다.

강사 김태민

TODAY'S GOAL

오늘 끝나면 할 수 있는 것

1. **코드가 실행되는 차례를 설명할 수 있다**

   동기와 비동기를 구분하고, call stack과 event loop로 실행 순서를 그려 봅니다.

2. **타이머로 시간이 걸리는 일을 다룰 수 있다**

   setTimeout · setInterval로 예약하고, clearTimeout · clearInterval로 멈춥니다.

3. **Promise와 async/await로 결과를 기다릴 수 있다**

   then · catch로 성공과 실패를 받고, await와 try…catch로 차례대로 씁니다.

오늘의 결과물

10초를 세는 플랭크 타이머 timer.js와 카페 주문을 처리하는 order.js

ROADMAP

오늘의 흐름

01

동기와 비동기

call stack
event loop

02

타이머

setTimeout
setInterval · 실습 01

실습 포함

03

Promise

콜백 · then · catch
Promise.all

04

async / await

await · try...catch
실행 순서 · 실습 02

실습 포함

앞의 두 섹션은 기다리는 동안 다음 줄로 넘어가는 비동기와 타이머, 뒤의 두 섹션은 기다린 결과를 받는 방법입니다.
마지막에는 섞인 코드의 실행 순서를 예측하고 실제 Console과 맞춰 봅니다.

되짚기 · 01

콜백함수를 이벤트에 등록하기

넘긴 곳	모양	부르는 때
DAY 14 forEach	`fruits.forEach((fruit) => console.log(fruit));`	바로 — 값마다 forEach가
DAY 16 addEventListener	`countBtn.addEventListener("click", addOne);`	누를 때마다 — 브라우저가
오늘 setTimeout	`setTimeout(sayHello, 2000);`	2초 뒤에 — 브라우저가

나중에 불리는 콜백 — 그동안 아래 줄이 먼저 실행됩니다

오늘은「시간이 지나면」불리는 콜백에서 시작해, 결과를 기다리는 Promise와 async/await까지 갑니다.

되짚기 · 02

화살표 함수 다시 읽기

```
// DAY 12 — { } 를 빼면 => 뒤의 식을 돌려줌
const sum = (a, b) => a + b;

// 오늘 자주 보는 모양 — 두 줄은 같은 일을 합니다
setTimeout(() => console.log("B"), 0);
setTimeout(function () { console.log("B"); }, 0);

// 매개변수가 둘인 화살표 함수 (섹션 03)
new Promise((resolve, reject) => {
  // 성공하면 resolve(값), 실패하면 reject(이유)
});
```

{ } 없는 한 줄

=> 뒤의 식을 그대로 돌려줍니다.
console.log 한 줄이면
'그 한 줄 실행'으로 읽습니다.

이름 없이 바로 넘김

변수에 담지 않고
부르는 곳의 괄호 안에
바로 써서 넘깁니다.

준비

오늘의 셋팅

```
Day17/                      ← 같은 저장소 안에 만듭니다
  index.html                ← 미리 받은 파일 (장표 예제)
  index.js                  ← 장표를 보며 함께 칩니다
  style.css                 ← 모두 함께 씁니다
  timer.html · timer.js ← 실습 01
  order.html · order.js ← 실습 02
$ git status
/* 오늘도 main에 바로 커밋합니다 */
```

예제는 index.js로

시간이 지나야 보이는 결과가
많아서 장표 예제는 index.js에
쓰고 index.html을 열어 봅니다.

찍힌 차례를 봅니다

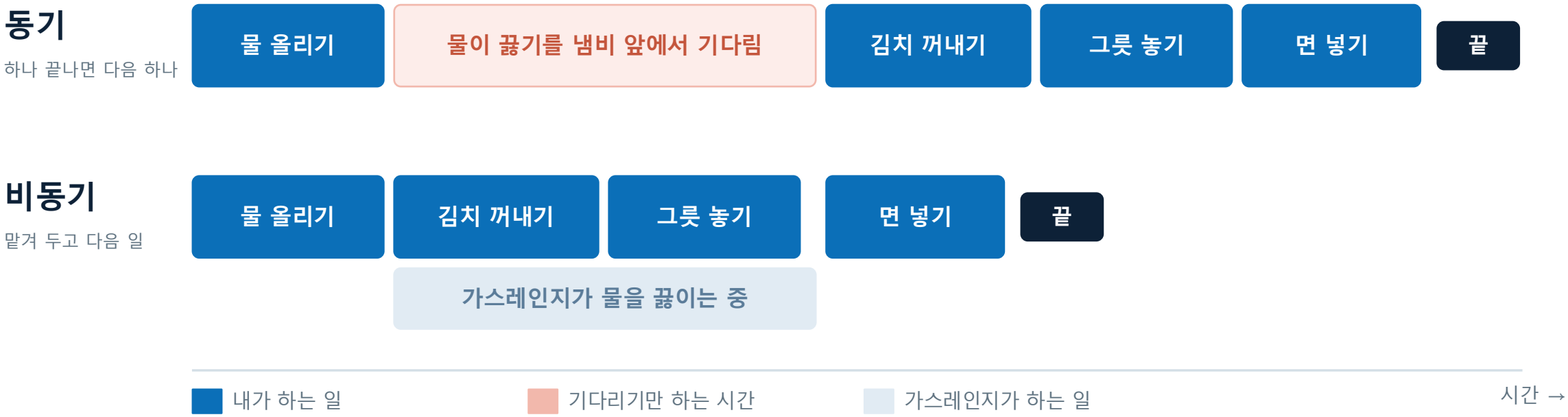
Console 줄 끝의
파일 이름:줄 번호로
어느 줄이 먼저인지 봅니다.

01

동기와 비동기

라면 끓이기 · 동기 · 비동기 · 웹에서의 동기 · 비동기 · call stack · event loop

라면 끓이기 — 물이 끓는 동안 할 일

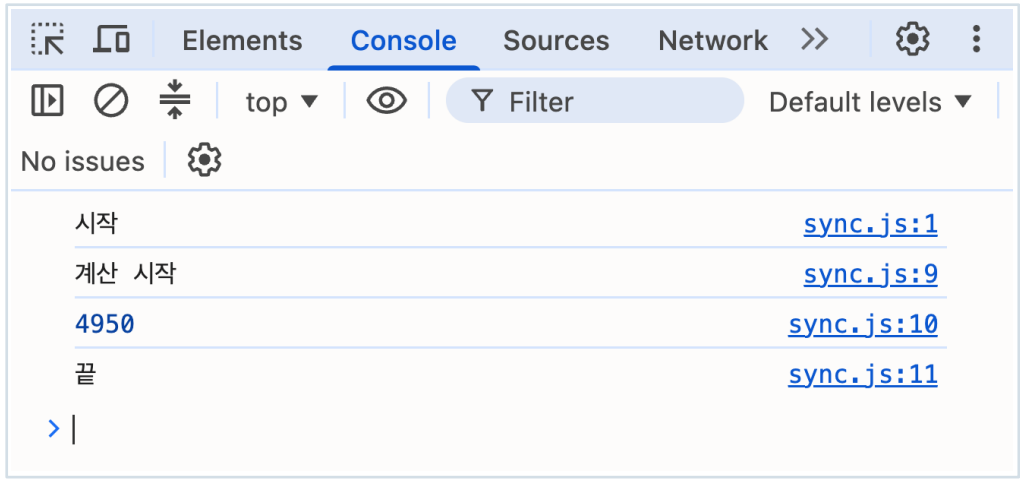


동기는 끝날 때까지 기다렸다가 다음 일, 비동기는 맡겨 두고 다음 일

물은 가스레인지가 끓이고, 나는 그동안 김치를 꺼내고 그릇을 놓습니다.

동기 — 앞 줄이 끝나야 실행되는 다음 줄

```
console.log("시작");
function longRunningTask() {
  let sum = 0;
  for (let i = 0; i < 100; i++) {
    sum += i;
  }
  return sum;
}
console.log("계산 시작");
console.log(longRunningTask());
console.log("끝");
```

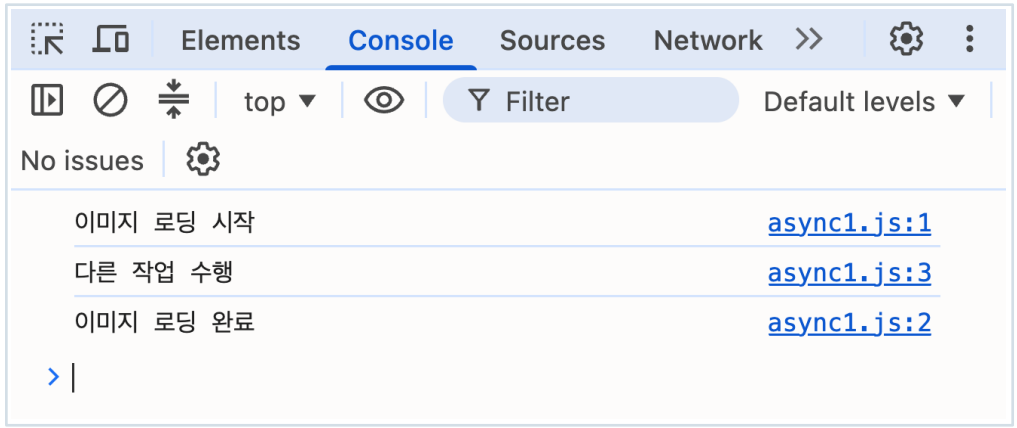


동기 — 차례대로

앞 줄이 끝나야 다음 줄
longRunningTask가 끝날 때까지
「끝」은 기다립니다

비동기 — 기다리지 않고 실행되는 다음 줄

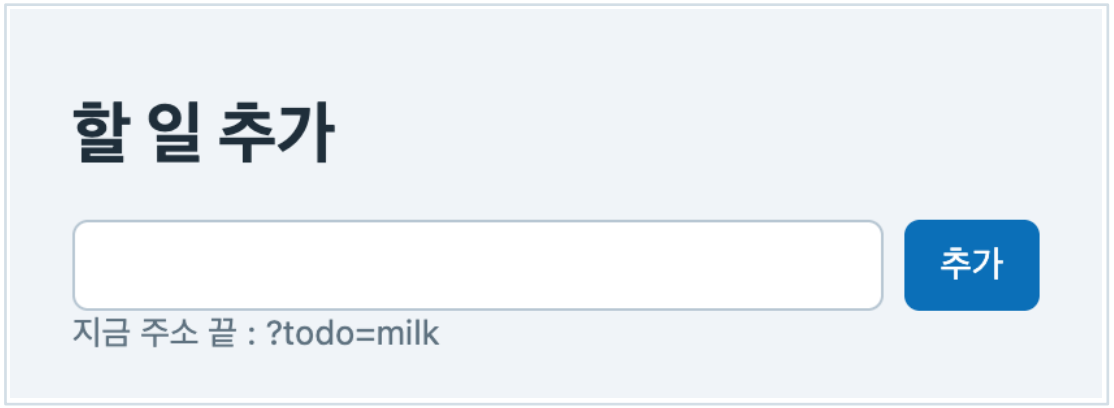
```
// index.js
console.log("이미지 로딩 시작");
setTimeout(() => console.log("이미지 로딩 완료"), 2000);
console.log("다른 작업 수행");
```



비동기 — 맡겨 두고 다음 줄

setTimeout은 '2초 뒤에 할 일'만 맡김
「다른 작업 수행」이 먼저 찍히고
2초 뒤 「이미지 로딩 완료」

웹에서 보는 동기와 비동기



DAY 16 — 제출하면 페이지 전체를 새로 받음



실습 02 — 페이지는 그대로, 목록만 바뀜

페이지를 통째로 새로 받으면 동기, 필요한 부분만 받아 바꾸면 비동기 통신

서버에서 데이터만 받아 오는 fetch는 DAY 18에서 — 오늘은 그 바탕인 「기다리는 동안 다음 줄」을 봅니다.

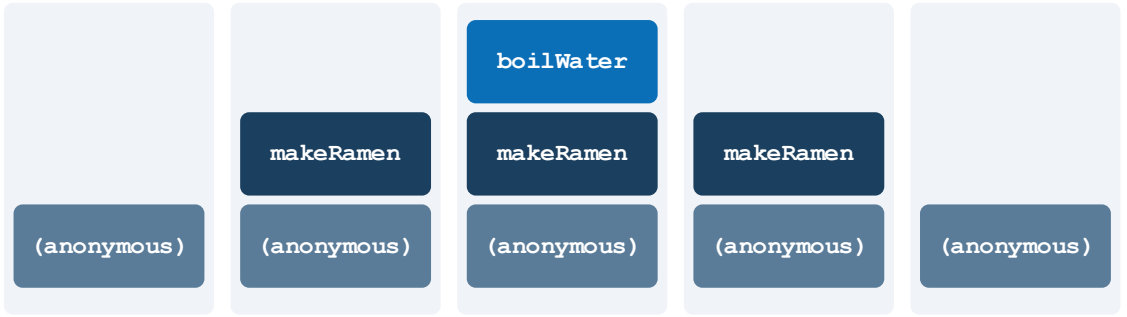
JavaScript는 한 번에 한 가지 — call stack

```
function boilWater() {
  debugger;   // 여기서 멈춰 call stack을 봅니다
  console.log("물 끓이기");
}

function makeRamen() {
  boilWater();
  console.log("라면 완성");
}

makeRamen();
```

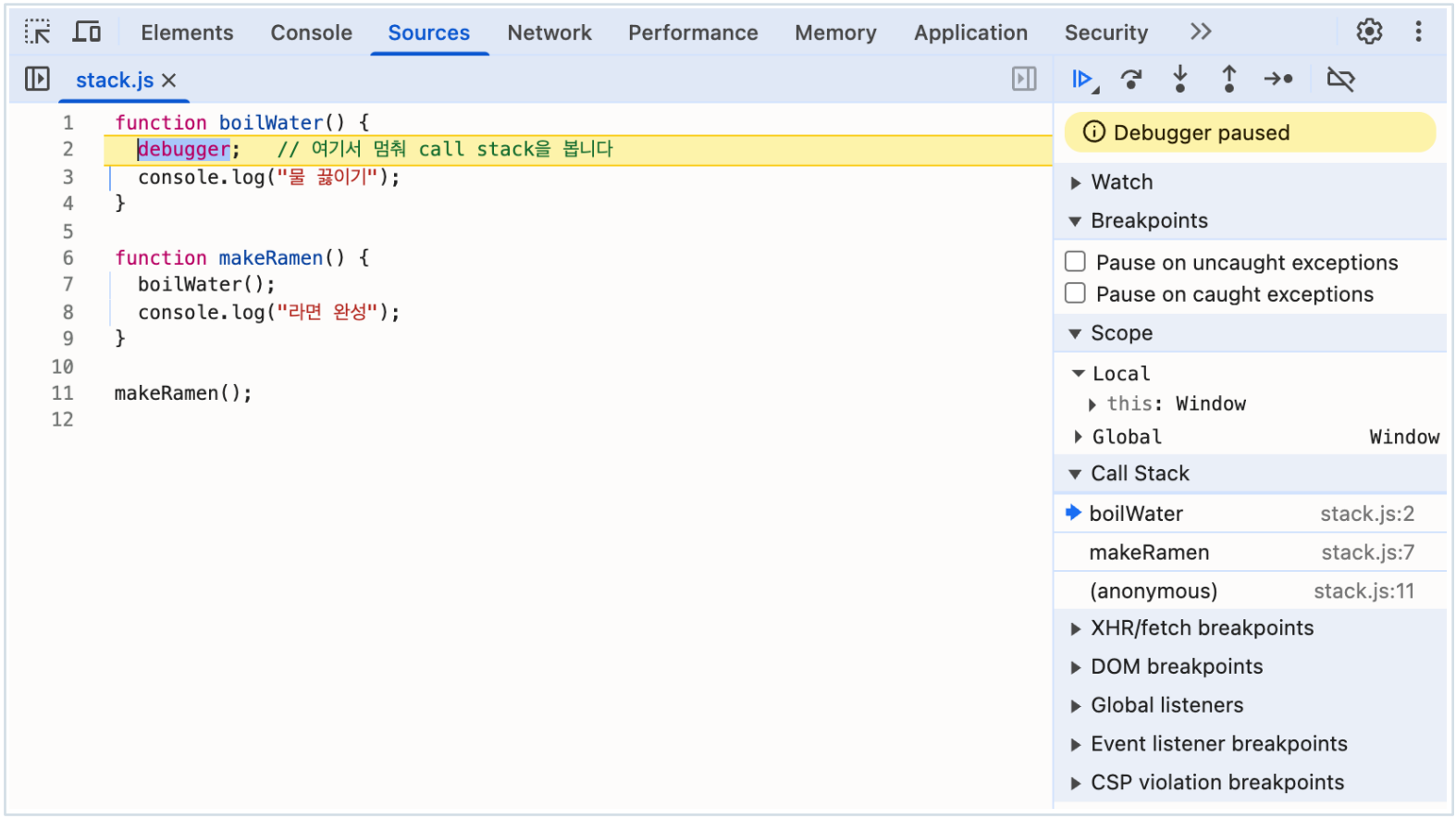
index.js를 실행하는 동안의 call stack



나중에 쌓인 함수가 먼저 끝납니다 — 접시를 쌓고 위에서부터 꺼내듯

맨 위의 함수가 끝나야 그 아래 함수가 이어서 실행됩니다. 한 번에 실행되는 것은 맨 위 하나입니다.

개발자 도구에서 call stack 직접 보기



Sources 패널 — 2번 줄에서 멈춘 stack.js와 오른쪽 Call Stack

읽는 법

노란 줄 — 멈춘 곳
맨 위 — 지금 실행 중인 함수
아래 — 먼저 부른 함수
▶ 버튼 — 이어서 실행

기다리는 일은 브라우저, 차례는 작업 큐

Call Stack

JavaScript가 지금 실행 중인 일

`console.log("C")`

`(anonymous) — index.js`

① 맡김 →

브라우저 — 타이머

시간을 세다가 다 되면 큐로

`setTimeout 0ms`

② 시간이 다 되면 ↓

작업 큐 (task queue)

차례를 기다리는 콜백

`() => console.log("B")`

← ③ 비면 꺼냄

세 가지 이름

call stack — 지금 실행 중인 일
작업 큐 — 차례를 기다리는 콜백
event loop — call stack이 비면 큐에서 하나씩 꺼내 실행

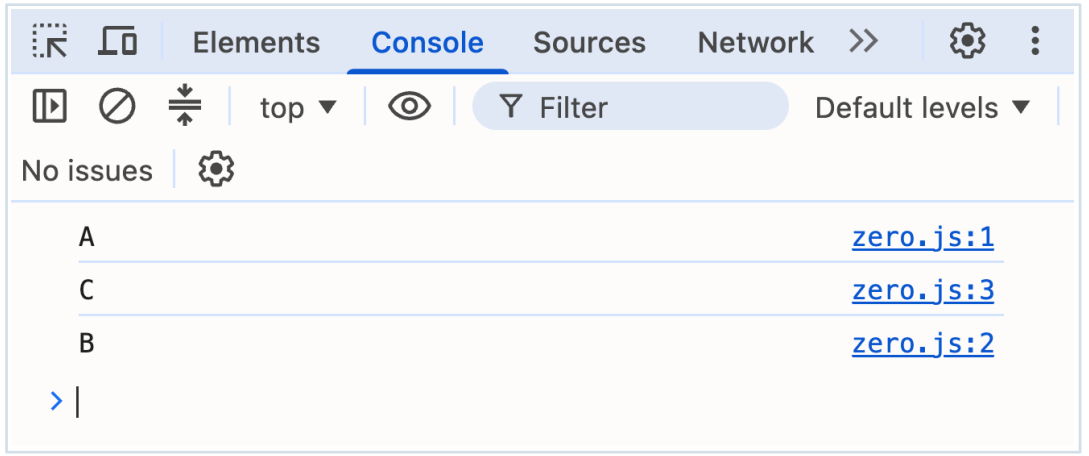
call stack이 비어야 큐의 콜백이 실행됩니다

그래서 setTimeout에 넘긴 함수는 아무리 짧게 예약해도 지금 실행 중인 코드가 다 끝난 뒤에 불립니다.

01 · 0ms

0ms로 예약해도 실행은 지금 코드가 끝난 뒤

```
console.log("A");
setTimeout(() => console.log("B"), 0);
console.log("C");
```



0ms — 「바로」가 아니라 지금 코드가 끝난 다음 차례

1 A를 찍음

1번 줄 — 바로 실행

2 브라우저에 맡김

2번 줄 — 콜백만 맡기고
바로 다음 줄로

3 C를 찍음

3번 줄 — 파일 끝
call stack이 비어 있음

4 B를 찍음

큐에서 기다리던 콜백을
event loop가 꺼내 실행

여기까지 정리

1	**동기**	앞의 일이 끝나야 다음 일 — 위에서 아래로 차례대로
2	**비동기**	오래 걸리는 일은 맡겨 두고, 기다리지 않고 다음 일
3	**call stack**	실행 중인 함수가 쌓이는 곳 — 나중에 쌓인 것이 먼저 끝남
4	**event loop**	call stack이 비면 큐에서 기다리던 콜백을 꺼내 실행
5	**setTimeout 0**	0ms여도 지금 코드가 다 끝난 뒤에 실행

다음은 시간을 정해 함수를 부르는 타이머를 자세히 봅니다

02

타이머

setTimeout · clearTimeout · setInterval · 괄호 실수 · 늦어지는 타이머 · 실습 01

'이 시간이 지나면 이 함수' 예약하기



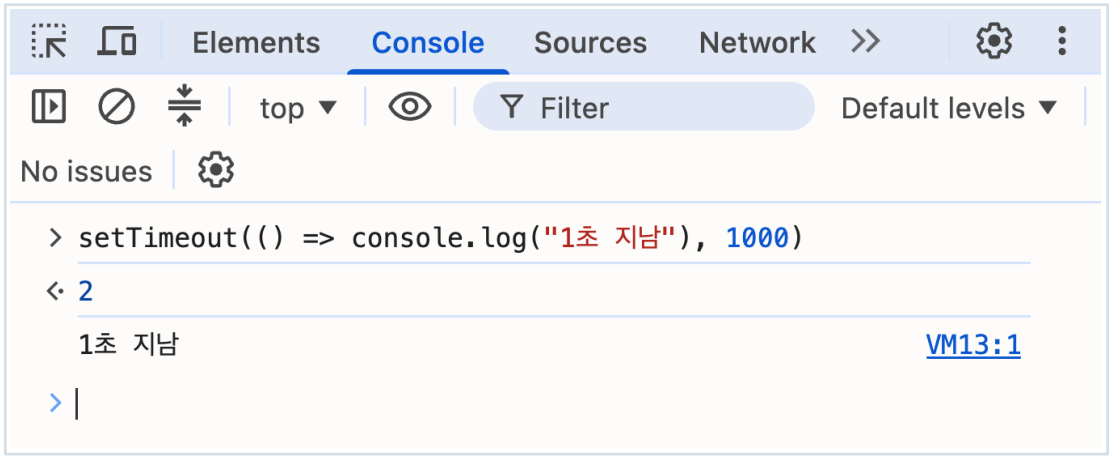
2000ms = 2초 — 시간이 지나면 브라우저가 sayHello를 한 번 불러 줍니다

setTimeout 자신은 예약만 하고 바로 끝나서, 다음 줄이 곧바로 실행됩니다.

바로 끝나고 번호를 돌려주는 setTimeout

```
// Console에 직접 칩니다
setTimeout(() => console.log("1초 지남"), 1000)

// < 2          ← 돌려준 번호 (그때그때 다름)
// 1초 지남     ← 1초 뒤에 찍힘
```



돌려준 번호

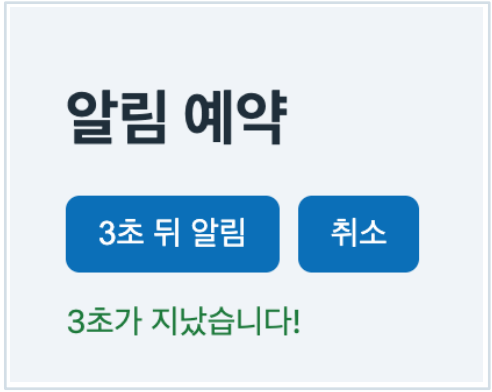
타이머를 구별하는 번호 (timeout ID)
취소할 때 이 번호를 씁니다
번호는 변수에 담아 둡니다

clearTimeout(번호) — 예약 취소

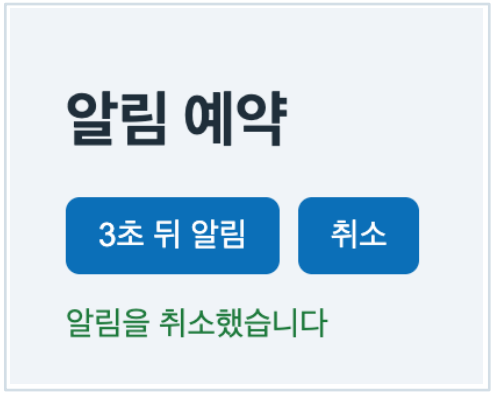
```
// index.js (HTML : 버튼 두 개와 p#msg)
const setBtn = document.querySelector("#set-btn");
const cancelBtn = document.querySelector("#cancel-btn");
const msgEl = document.querySelector("#msg");
let timeoutId = null;

setBtn.addEventListener("click", () => {
  msgEl.textContent = "3초 뒤에 알려 드립니다";
  timeoutId = setTimeout(() => {
    msgEl.textContent = "3초가 지났습니다!";
  }, 3000);
});

cancelBtn.addEventListener("click", () => {
  clearTimeout(timeoutId);
  msgEl.textContent = "알림을 취소했습니다";
});
```



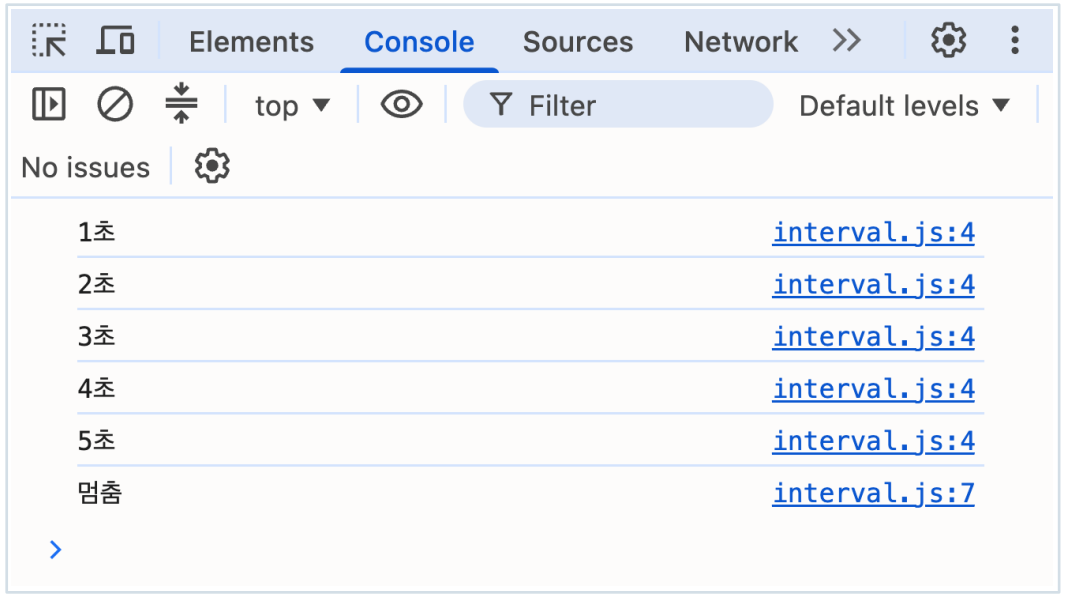
「3초 뒤 알림」을 누르고 3초 뒤



누르고 1초 뒤 취소 — 3초가 지나도 그대로

setInterval — 정해진 시간마다 되풀이

```
// index.js
let count = 0;
const intervalId = setInterval(() => {
  count += 1;
  console.log(`${count}초`);
  if (count === 5) {
    clearInterval(intervalId);
    console.log("멈춤");
  }
}, 1000);
```



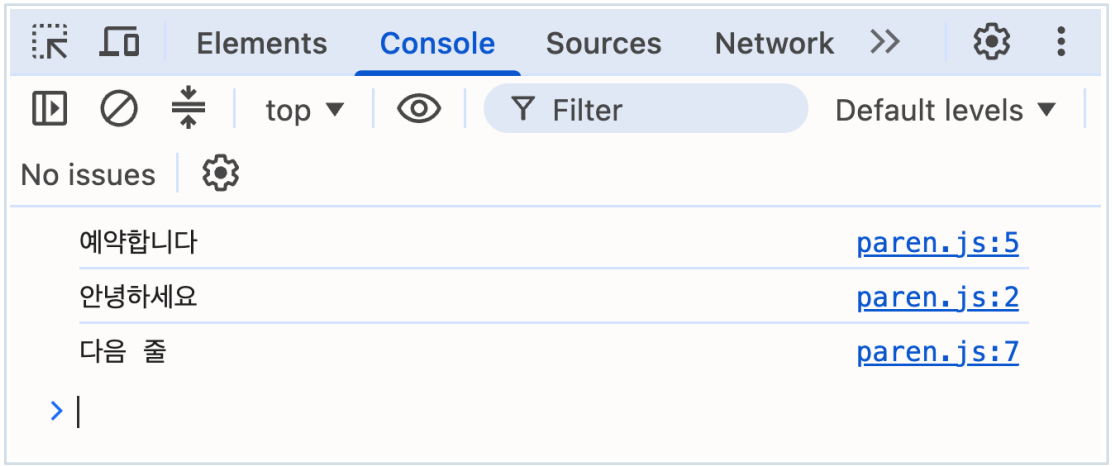
setInterval과 clearInterval

첫 실행도 1초 뒤 — 바로가 아님
돌려준 번호를 clearInterval에 넣어 멈춤

함수 뒤에 괄호를 붙이면 지금 실행

```
// index.js
function sayHello() {
  console.log("안녕하세요");
}

console.log("예약합니다");
setTimeout(sayHello(), 2000);    // 괄호를 붙임
console.log("다음 줄");
```



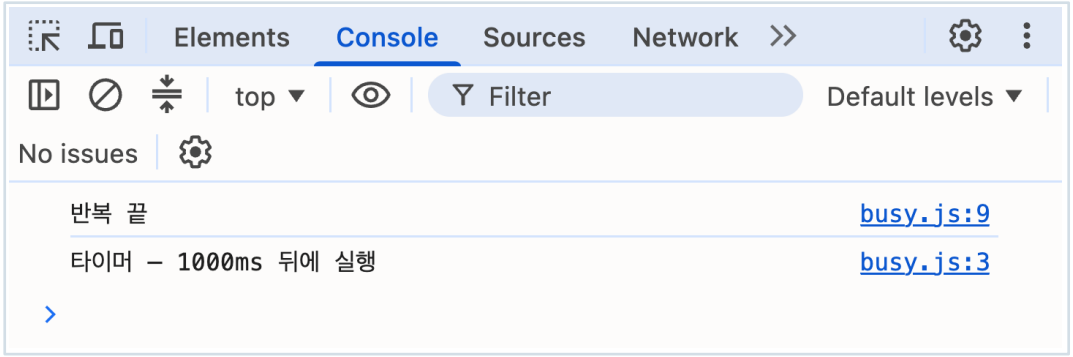
sayHello와 sayHello()

sayHello — 함수를 넘김 (2초 뒤 불림)
sayHello() — 지금 실행하고 결과를 넘김
DAY 16 11장의 addOne()과 같은 실수

정한 시간은 '최소한' — 앞의 일이 끝나야 실행

```
// index.js
const start = Date.now();   // 지금 시각 (ms)
setTimeout(() => {
  console.log(`타이머 — ${Date.now() - start}ms 뒤에 실행`);
}, 0);

while (Date.now() - start < 1000) {
  // 1초 동안 아무것도 안 하고 붙잡고 있기
}
console.log("반복 끝");
```



0ms로 예약했는데

반복이 1초 동안 call stack을 붙잡아
타이머는 반복이 끝난 뒤 실행
그동안은 클릭도 처리하지 못함

실습 01

플랭크 타이머 만들기

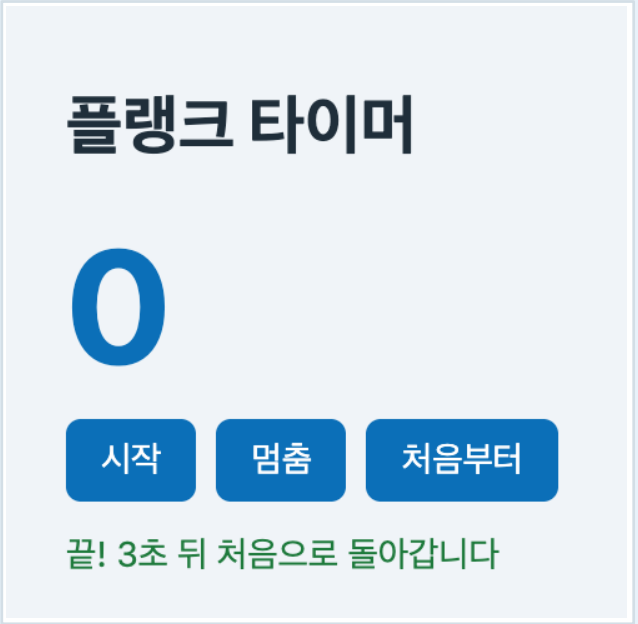
timer.html은 그대로 두고 timer.js만 써서, 10초를 거꾸로 세는 타이머를 만듭니다.
setInterval · clearInterval · setTimeout과 DAY 16의 addEventListener를 함께 씁니다.

만들 것

① render — 남은 초를 화면에 쓰기
② reset — 처음 상태로 되돌리기
③ tick — 하나 줄이고, 0이면 멈춤
④ 시작 — 돌고 있으면 또 시작 안 함
⑤ 멈춤 — 남은 시간은 그대로
⑥ 처음부터 — reset 연결



시작을 누르고 3초 뒤



10초 뒤 — 0과 끝 문구

실습 01 · 확인

빨간 줄 없이 화면으로 나타나는 타이머 실수

바꿀 곳	결과
① 31번 줄 clearInterval(timerId); 앞에 // 붙이기	0에서 멈추지 않고 -1, -2 … 10으로 돌아가도 계속 줄어듦
② 40번 줄의 timerId !== null || 지우기	시작을 두 번 누르면 1초에 2씩 — 멈춤을 눌러도 계속 줄어듦
③ 43번 줄 tick을 tick()으로	누르자마자 9가 되고 더 줄지 않음 — 빨간 줄 없음
④ 49번 줄 timerId = null; 앞에 // 붙이기	멈춘 뒤 시작을 눌러도 다시 가지 않음 — 처음부터는 됨

타이머가 이상하면 '두 개가 돌고 있는지, 번호를 지웠는지'를 봅니다

여기까지 정리

1	**setTimeout**	정한 시간이 지나면 함수를 한 번 — 예약하고 바로 다음 줄
2	**타이머 번호**	setTimeout · setInterval이 돌려줌 — 취소할 때 씀
3	**setInterval**	정한 시간마다 되풀이 — clearInterval(번호)로 멈춤
4	**괄호 없이**	setTimeout(sayHello, 1000) — 괄호를 붙이면 지금 실행
5	**시간은 최소한**	앞의 코드가 call stack을 비워야 실행

다음은 기다린 결과를 받는 방법 — 콜백에서 Promise로

03

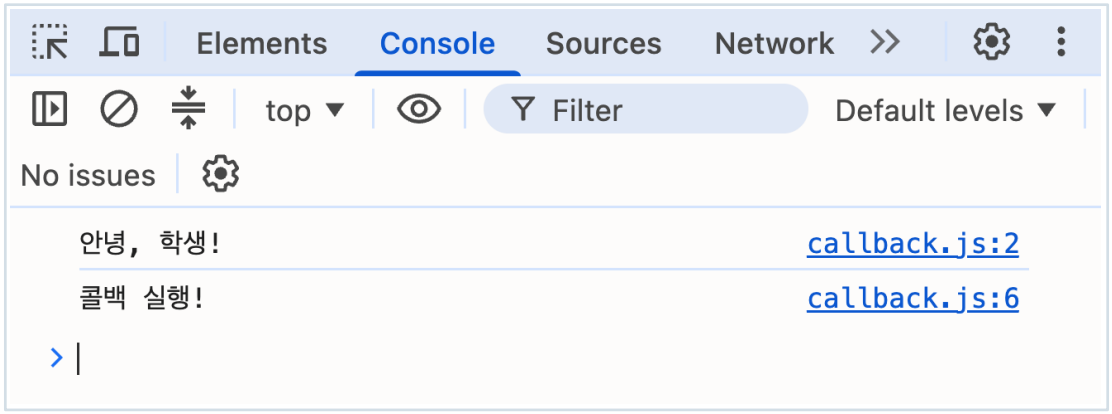
Promise

콜백 · 콜백 지옥 · 세 가지 상태 · resolve · reject · then · catch · Promise.all

콜백 — 다른 함수에 넘겨 그 함수가 부르는 함수

```
// index.js
function sayHello(name, callback) {
  console.log(`안녕, ${name}!`);
  callback();
}

sayHello("학생", () => console.log("콜백 실행!"));
```

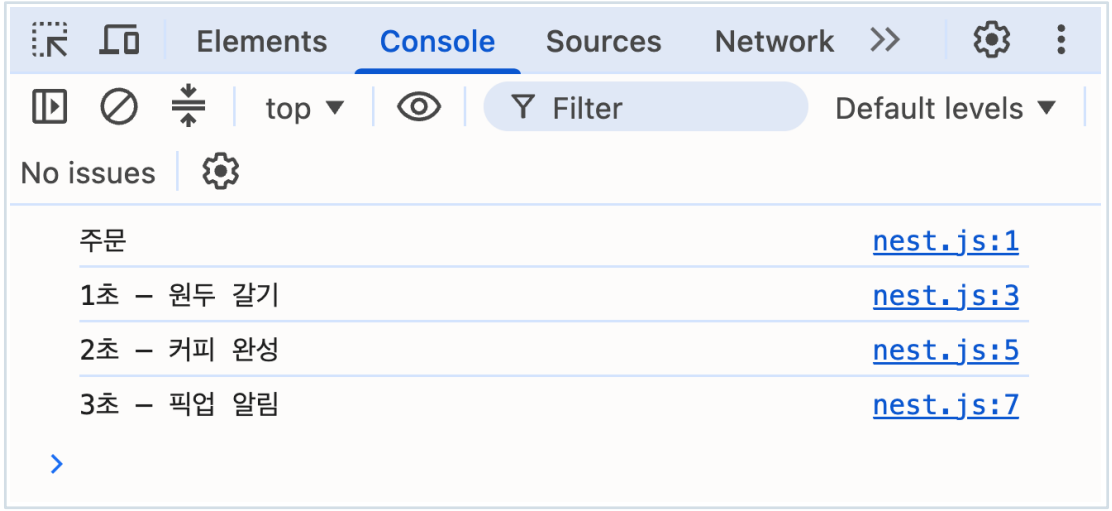


1기 자료의 예

sayHello가 인사를 찍고
넘겨받은 callback을 부릅니다
타이머 · 이벤트도 이 모양

차례로 기다릴 일이 늘수록 깊어지는 들여쓰기

```
// index.js
console.log("주문");
setTimeout(() => {
  console.log("1초 — 원두 갈기");
  setTimeout(() => {
    console.log("2초 — 커피 완성");
    setTimeout(() => {
      console.log("3초 — 픽업 알림");
    }, 1000);
  }, 1000);
}, 1000);
```



콜백 지옥

단계마다 한 칸씩 안으로 들어감
실패 처리도 단계마다 따로
→ Promise로 아래로 이어 씀

Promise — 나중에 나올 결과를 담는 객체

pending

기다리는 중 — 처음 상태

주문하고 진동벨을 받아
기다리는 상태

→ resolve(값)

fulfilled

이행 — 음료가 나옴

값은 then에 넘긴 함수가 받음

→ reject(이유)

rejected

거부 — 품절

이유는 catch에 넘긴 함수가 받음

settled

결과가 정해진 상태
한 번 정해지면 바뀌지 않음

처음에는 pending, 한 번 정해지면 fulfilled나 rejected에서 바뀌지 않습니다

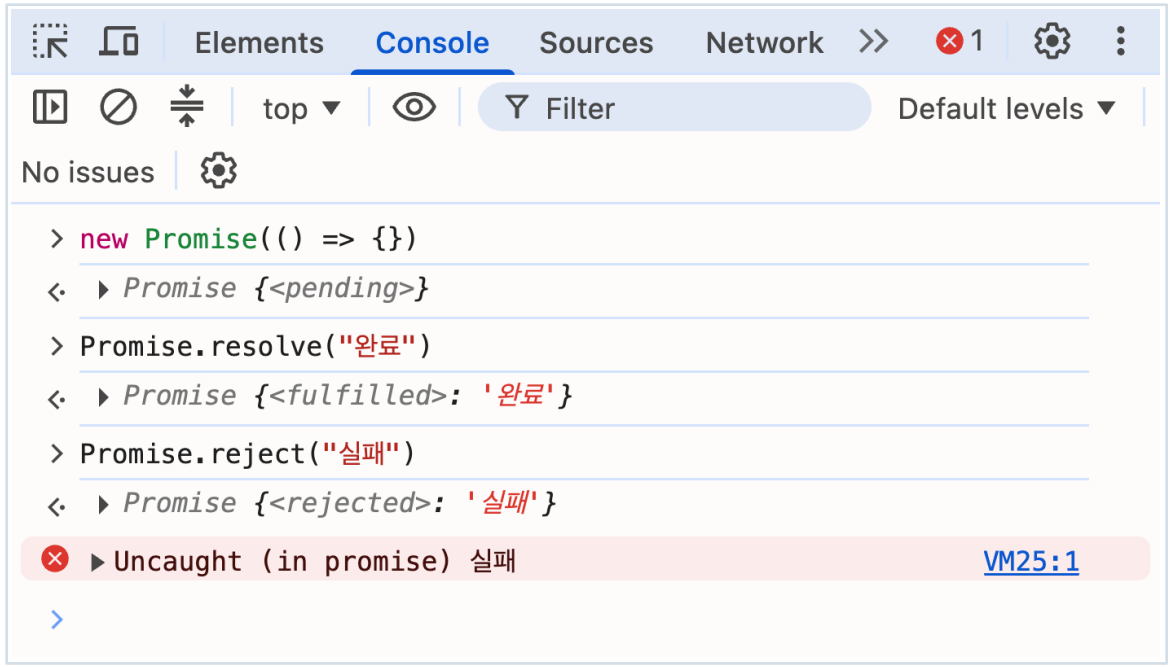
결과는 then(이행)과 catch(거부)에 넘긴 함수로 받습니다 — 진동벨이 울리면 음료를 받듯이.

Console에서 보는 세 가지 상태

```
// Console에 한 줄씩 칩니다
new Promise(() => {})
// resolve도 reject도 부르지 않음 → pending

Promise.resolve("완료")
// 이행된 Promise를 바로 만듦 → fulfilled

Promise.reject("실패")
// 거부된 Promise를 바로 만듦 → rejected
```



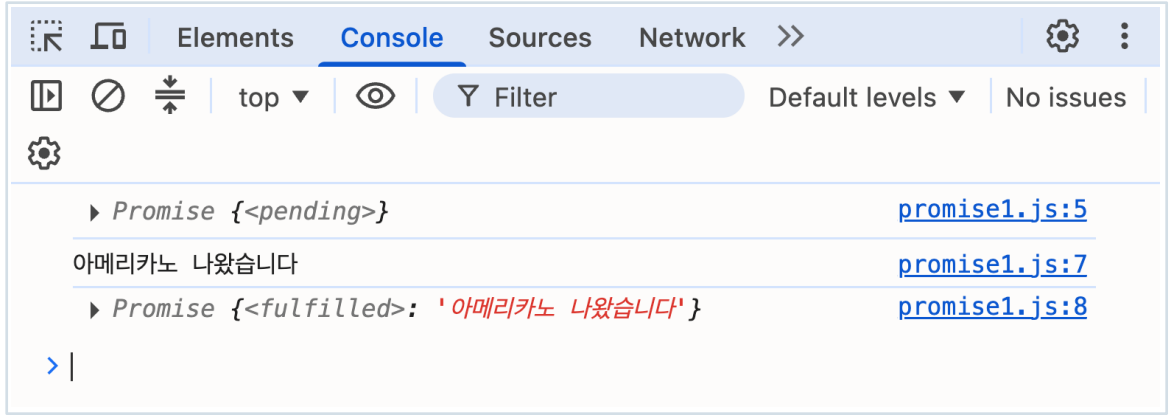
꺾쇠 안이 상태

<pending> · <fulfilled> · <rejected>
콜론 뒤는 결과 값
받는 곳 없는 거부는 빨간 줄

new Promise로 만들고, then으로 결과 받기

```
// index.js
const order = new Promise((resolve, reject) => {
  setTimeout(() => resolve("아메리카노 나왔습니다"), 2000);
});

console.log(order);
order.then((message) => {
  console.log(message);
  console.log(order);
});
```



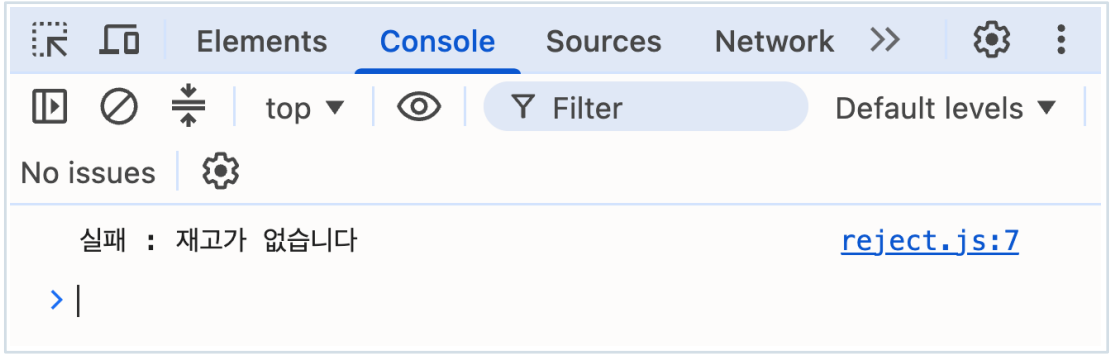
읽는 법

resolve(값)를 부르면 이행 — fulfilled
then의 함수가 그 값을 받음
만든 직후에는 아직 pending

실패는 reject, 받는 곳은 catch

```
// index.js
const order = new Promise((resolve, reject) => {
  setTimeout(() => reject("재고가 없습니다"), 1000);
});

order
  .then((message) => console.log("성공 :", message))
  .catch((error) => console.log("실패 :", error));
```

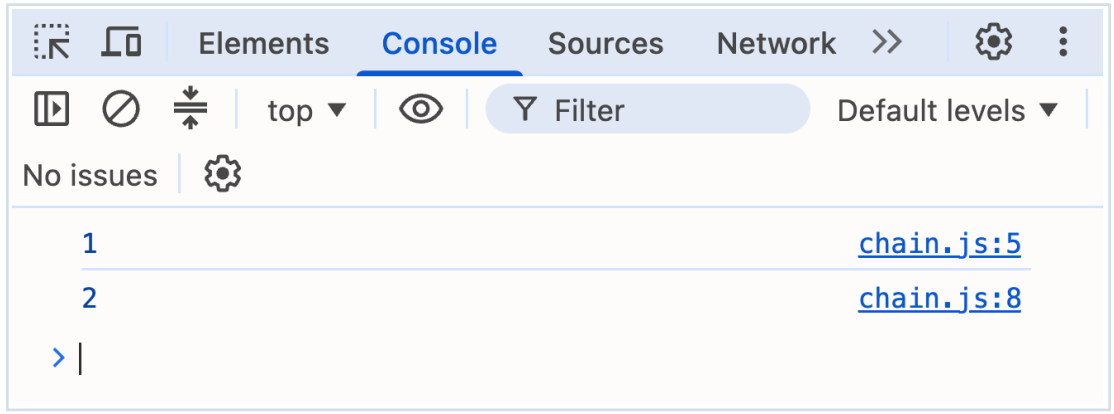


읽는 법

reject(이유)를 부르면 거부 — rejected
then은 건너뛰고 catch가 받음
이유는 catch 함수의 매개변수로

then 뒤에 then 이어 붙이기

```
// index.js
new Promise((resolve) => {
  setTimeout(() => resolve(1), 1000);
})
  .then((result) => {
    console.log(result);
    return result + 1;
  })
  .then((result) => console.log(result))
  .catch((error) => console.log("에러:", error));
```



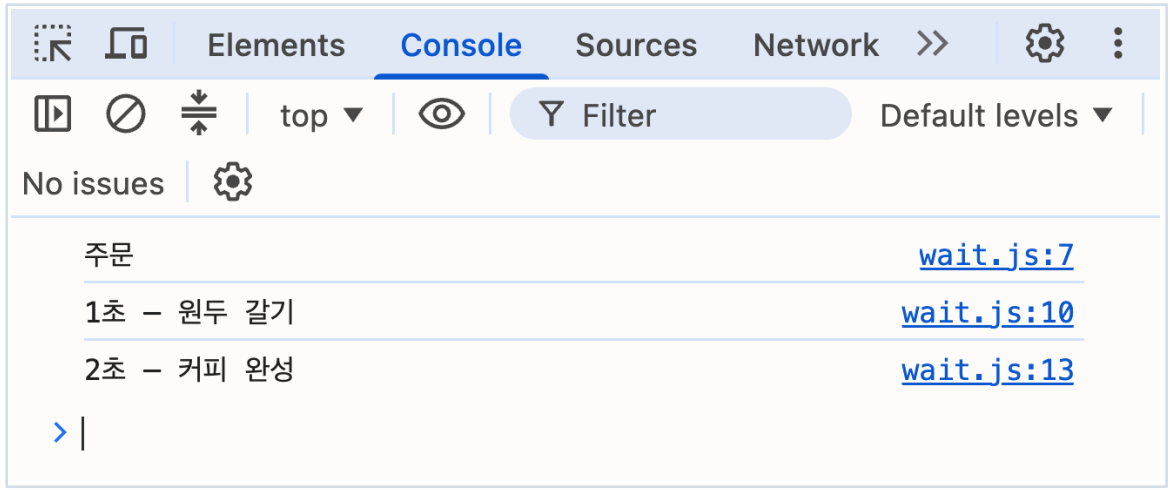
읽는 법

then은 새 Promise를 돌려주고
return한 값이 다음 then으로
실패는 맨 끝의 catch 하나로

기다리는 함수를 만들어 차례로 이어 쓰기

```
// index.js
function wait(ms) {
  return new Promise((resolve) => {
    setTimeout(resolve, ms);
  });
}

console.log("주문");
wait(1000)
  .then(() => {
    console.log("1초 — 원두 갈기");
    return wait(1000);
  })
  .then(() => console.log("2초 — 커피 완성"));
```



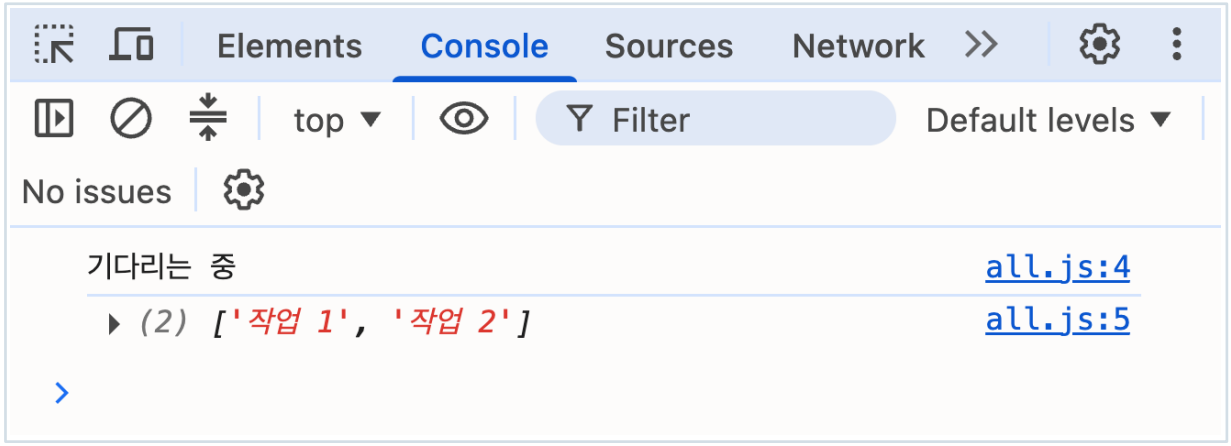
29장과 비교

안으로 들어가지 않고 아래로
then 안에서 Promise를 return하면
그것이 끝난 뒤 다음 then

Promise.all — 여러 개를 함께 기다리기

```
// index.js
const p1 = new Promise((resolve) => setTimeout(() => resolve("작업 1"), 1000));
const p2 = new Promise((resolve) => setTimeout(() => resolve("작업 2"), 2000));

console.log("기다리는 중");
Promise.all([p1, p2]).then((results) => console.log(results));
```



읽는 법

모두 이행되면 결과를 배열로
1초 · 2초짜리 → 2초 뒤 한 번에
하나라도 거부되면 바로 거부

여기까지 정리

1	**콜백**	다른 함수에 넘겨 그 함수가 부르는 함수 — 겹치면 콜백 지옥
2	**Promise**	나중에 나올 결과를 담는 객체 — pending · fulfilled · rejected
3	**resolve · reject**	성공하면 resolve(값), 실패하면 reject(이유)
4	**then · catch**	이행되면 then, 거부되면 catch — 이어 붙일 수 있음
5	**Promise.all**	여러 Promise를 함께 기다려 결과를 배열로

다음은 then 없이 '기다렸다가 다음 줄'로 쓰는 async / await

04

async / await와 실행 순서

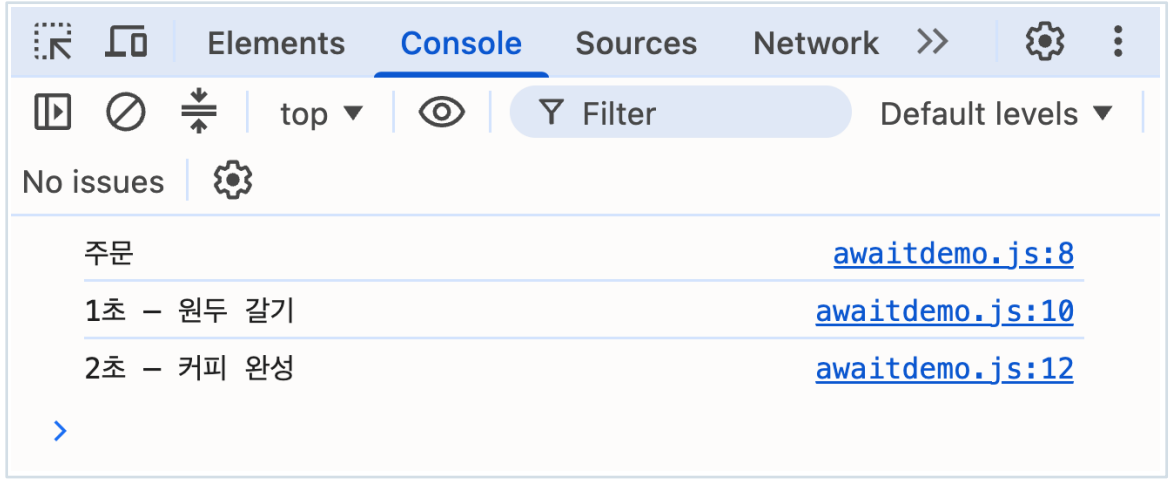
async · await · try...catch · await는 async 안에서 · 실행 순서 예측 · 실습 02

await — Promise가 끝나야 실행되는 다음 줄

```
// index.js
function wait(ms) {
  return new Promise((resolve) => {
    setTimeout(resolve, ms);
  });
}

async function makeCoffee() {
  console.log("주문");
  await wait(1000);
  console.log("1초 – 원두 갈기");
  await wait(1000);
  console.log("2초 – 커피 완성");
}

makeCoffee();
```



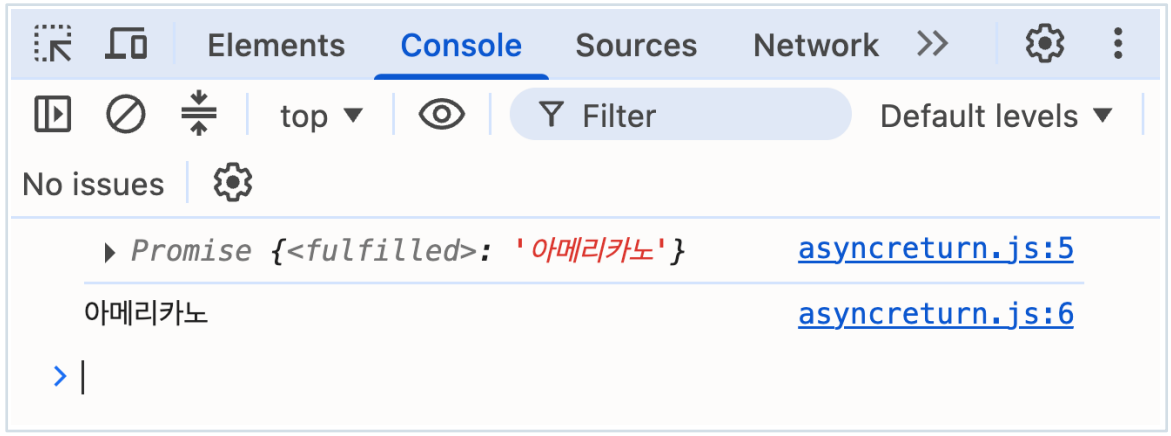
35장과 같은 결과

then 대신 await로 아래로 차례대로
await는 async 함수 안에서
멈추는 것은 그 함수 안뿐

언제나 Promise를 돌려주는 async 함수

```
// index.js
async function getMenu() {
  return "아메리카노";
}

console.log(getMenu());
getMenu().then((menu) => console.log(menu));
```



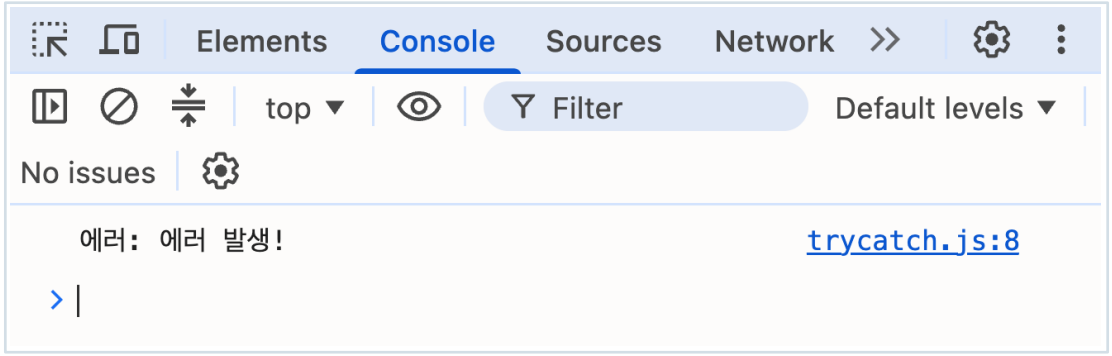
읽는 법

return한 값은 Promise에 담겨 나옴
값을 꺼내려면 then이나 await
실습 02의 makeDrink가 이 모양

실패를 받는 try...catch

```
// index.js
async function fetchWithError() {
  try {
    await new Promise((resolve, reject) => {
      setTimeout(() => reject("에러 발생!"), 1000);
    });
    console.log("여기는 실행되지 않습니다");
  } catch (error) {
    console.log("에러:", error);
  }
}

fetchWithError();
```



읽는 법

await한 Promise가 거부되면
그 줄에서 오류가 던져지고 catch로
「여기는 실행되지 않습니다」는 안 찍힘

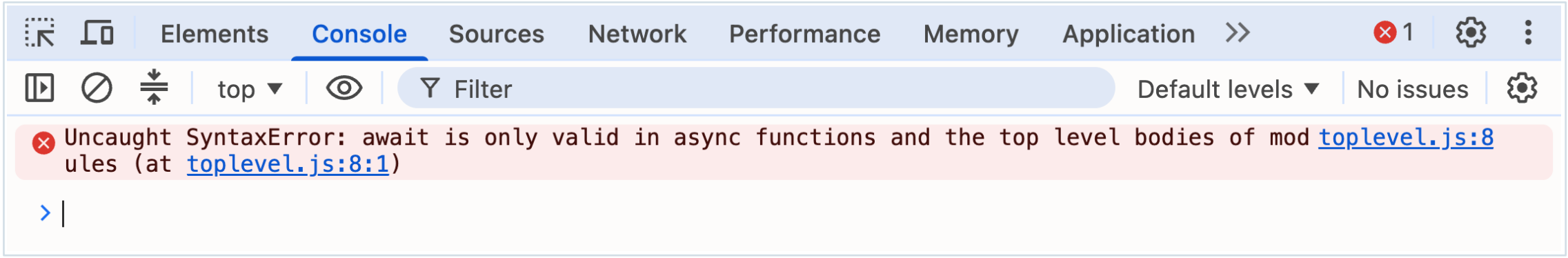
async 함수 안에서 쓰는 await

```
// index.js
function wait(ms) {
  return new Promise((resolve) => {
    setTimeout(resolve, ms);
  });
}

console.log("시작");
await wait(1000);
console.log("1초 뒤");
```

읽는 법

함수 밖에서 await → SyntaxError
파일 전체가 실행되지 않아
「시작」도 찍히지 않음

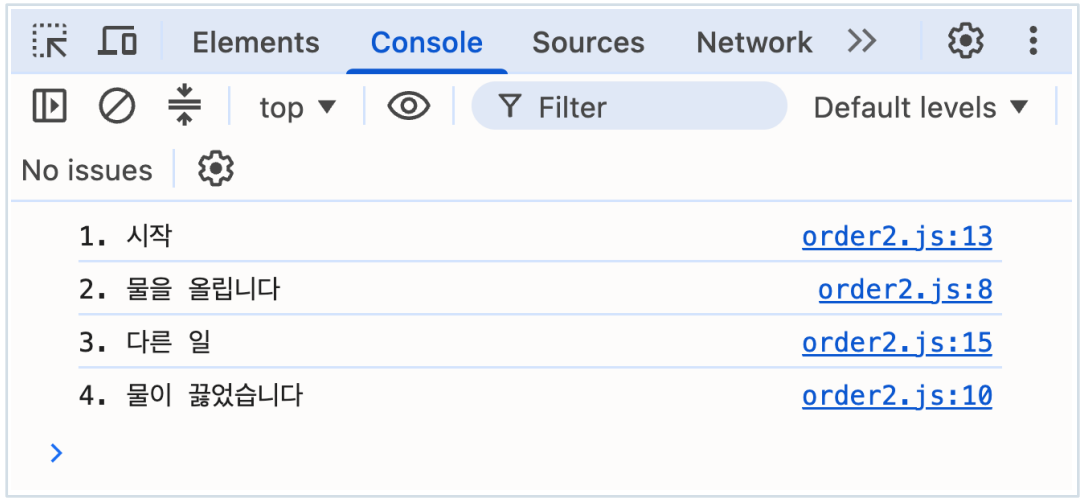


await 앞까지는 바로, 뒤는 기다린 다음

```
// index.js
function wait(ms) {
  return new Promise((resolve) => {
    setTimeout(resolve, ms);
  });
}

async function boil() {
  console.log("2. 물을 올립니다");
  await wait(1000);
  console.log("4. 물이 끓었습니다");
}

console.log("1. 시작");
boil();
console.log("3. 다른 일");
```

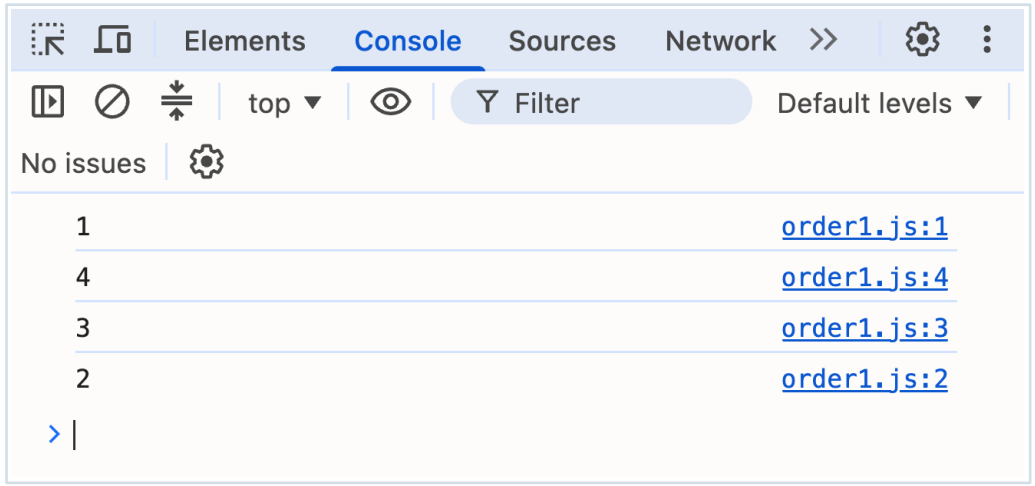


차례

1 → boil() 안의 2까지 바로
await에서 boil만 멈춤 → 밖의 3
1초 뒤 4

Promise의 then이 setTimeout 0보다 먼저

```
console.log("1");
setTimeout(() => console.log("2"), 0);
Promise.resolve().then(() => console.log("3"));
console.log("4");
```



큐가 둘 — Promise의 큐를 먼저 다 비웁니다

1 지금 코드 — 1, 4

1번 줄과 4번 줄
call stack에서 바로 실행

2 마이크로태스크 — 3

Promise의 then 콜백
지금 코드가 끝나면 먼저

3 작업 큐 — 2

setTimeout의 콜백
마이크로태스크를 다 비운 뒤

실습 02

카페 주문 처리하기

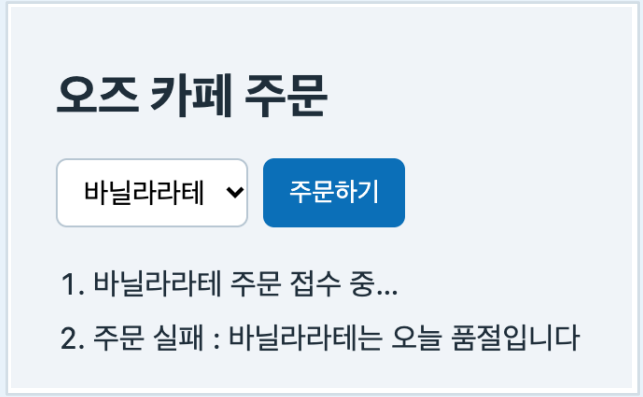
order.html은 그대로 두고 order.js만 써서, 주문을 받고 음료를 만들어 픽업을 안내합니다.
Promise · resolve · reject · async · await · try…catch와 DAY 16의 submit을 함께 씁니다.

만들 것

① addLog — 진행 상황 한 줄 더하기
② wait — ms만큼 기다리는 Promise
③ takeOrder — 품절이면 reject
④ makeDrink — 1.5초 뒤 완성 글
⑤ serve — await로 차례로, catch
⑥ 주문하기 submit — 새로고침 막기



아메리카노 · 카페라테를 잇달아 주문



바닐라라테 — 품절이라 catch로

실습 02

일부러 틀려 보기

완성한 order.js를 한 곳씩 바꿔 저장하고, 결과가 어떻게 바뀌는지 본 뒤 되돌립니다.
①은 빨간 줄이 0.5초 뒤에, ④는 열자마자 뜹니다. ②와 ③은 화면을 보고 찾습니다.

바꿀 곳	결과
① 44번 줄의 await 빼기 — takeOrder(menu);	품절인데 확인 · 완성이 찍히고, Console에 Uncaught (in promise)
② 46번 줄의 await 빼기 — makeDrink(menu);	기다리지 않고 바로「 [object Promise]! 픽업대로 오세요」
③ 16번 줄의 return 빼기 — new Promise(...	확인 다음 1.5초를 기다리지 않고 바로 완성
④ 41번 줄의 async 빼기	열자마자 44번 줄 SyntaxError — 주문하면 새로 열림

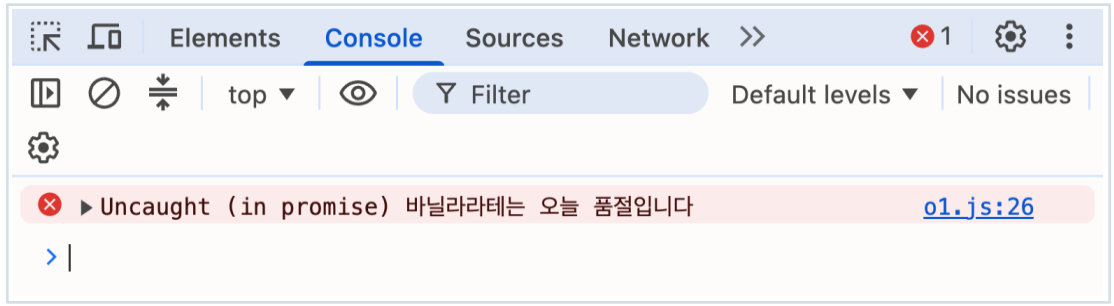
네 가지를 다 보고 되돌렸으면 커밋합니다 — 실습 02 끝

await를 빠뜨리면 — 품절인데도 나오는 완성 안내

```
// order.js — serve (일부러 틀린 곳 ①)
async function serve(menu) {
  addLog(`${menu} 주문 접수 중…`);
  try {
    takeOrder(menu);      // await를 빠뜨림
    addLog(`${menu} 확인 — 만드는 중…`);
    const drink = await makeDrink(menu);
    addLog(`${drink}! 픽업대로 오세요`);
  } catch (error) {
    addLog(`주문 실패 : ${error}`);
  }
}
```

확인하는 법

순서가 이상하거나 [object Promise]가 보이면
Promise를 돌려주는 함수 앞에 await가 있는지 봅니다.



Console — 0.5초 뒤, 받는 곳 없는 거부



화면 — 품절인데 확인 · 완성까지

점검

비동기에서 자주 나오는 세 가지

!

타이머가 이상해요

두 개가 돌고 있거나
clearInterval이 빠졌습니다.
멈춘 뒤 번호를 null로 했는지도.

!

[object Promise]가 보여요

Promise를 돌려주는 함수 앞에
await가 빠졌습니다.
await는 async 함수 안에서.

!

await에서 SyntaxError

await를 쓴 함수 앞에
async가 없습니다.
함수 밖에서도 안 됩니다.

‘언제 실행되는가’를 Console의 순서와 줄 번호로 확인합니다

찍힌 줄 끝의 파일 이름:줄 번호를 보면 어느 줄이 언제 실행됐는지 알 수 있습니다.

WRAP UP

오늘 정리

개념

동기는 차례대로, 비동기는 맡겨 두고 다음 줄

call stack이 비어야 큐의 콜백이 실행됨

타이머의 시간은「최소한」

Promise — pending에서 fulfilled · rejected로

async 함수는 언제나 Promise를 돌려줌

await 앞까지는 바로, 뒤는 기다린 다음

직접 쓸 수 있어야 하는 것

setTimeout · setInterval로 예약하고 멈추기

new Promise로 resolve · reject 부르기

then · catch로 결과와 실패 받기

async / await와 try...catch로 차례대로

10초 타이머와 카페 주문 처리

Console 순서로 실행 순서 확인하기

참고교재

교재에서 함께 읽을 곳

오늘 수업과 이어지는 장입니다. 복습할 때 장 번호로 찾아 읽습니다.

길벗캠퍼스

『백엔드 입문자를 위한 모던 자바스크립트 & Node.js』

이창현 지음

9장. ES2015+ 문법

위키북스

『모던 자바스크립트 Deep Dive』

이웅모 지음

38장. 브라우저의 렌더링 과정
41장. 타이머
42장. 비동기 프로그래밍

NEXT

DAY 18

fetch API, JSON, 오류 처리

fetch로 서버에서 데이터를 받아 화면에 그리고,
try · catch · finally로 실패까지 다룹니다.

수고하셨습니다.